

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0304	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	A. Ashriktha Reddy	Reg. No.:	192424172
Section / Batch:	2024	Date:	13.07.2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 1

Section 1: Regular Expressions – Resume Information Extraction

Code:

```
import re

resume = """
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume).group(1)
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume).group()
mobile = re.search(r"\b\d{10}\b", resume).group()
skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", resume, re.I)
experience = int(re.search(r"(\d+)\s+years", resume).group(1))

print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", skills)
print("Experience:", experience, "Years")

if experience >= 2 and "Python" in skills:
    print("Status: Eligible")
else:
    print("Status: Not Eligible")
```

Explanation:

- Uses Regular Expressions to extract name, email, mobile number, skills, and experience.
- Checks whether the candidate has Python skill and minimum 2 years of experience.
- Displays the candidate profile and eligibility status.

OUTPUT:



```
Python 3.7.6 Shell In: 100
> In [1]: import re
> resume = """
> Name: Rahul Sharma
> Email: rahul@gmail.com
> Mobile: 9876543210
> Skills: Python, Java, SQL, Machine Learning, NLP
> Experience: 3 years
> """
> name = re.search(r"Name:\s*(.*)", resume).group(1)
> email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume).group()
> mobile = re.search(r"\b\d{10}\b", resume).group()
> skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", resume, re.I)
> experience = int(re.search(r"(\d+)\s+years", resume).group(1))
> print("Name:", name)
> print("Email:", email)
> print("Mobile:", mobile)
> print("Skills:", skills)
> print("Experience:", experience, "Years")
> if experience >= 2 and "Python" in skills:
>     print("Status: Eligible")
> else:
>     print("Status: Not Eligible")
>
Out[1]: Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 Years
Status: Eligible
```

Section 2: Regular Expressions – Product Search System

Code:

```
import re
```

```
products = ["Laptop", "Laptop Bag", "Gaming Laptop", "Mouse", "Wireless Mouse",  
"Keyboard"]
```

```
print("Exact Match:")  
print([p for p in products if re.fullmatch("Laptop", p)])
```

```
print("Prefix Match:")  
print([p for p in products if re.match("La", p)])
```

```
print("Suffix Match:")  
print([p for p in products if re.search("Bag$", p)])
```

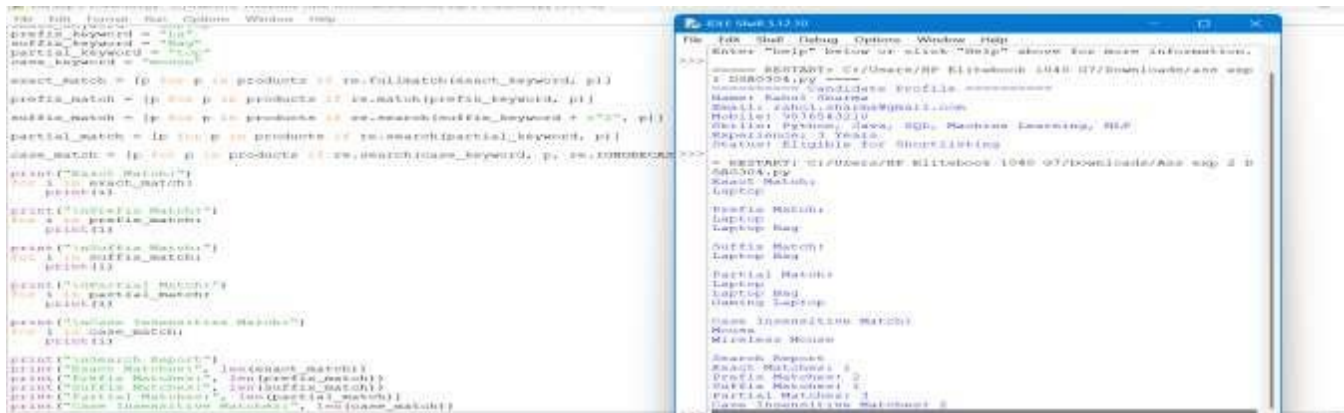
```
print("Partial Match:")  
print([p for p in products if re.search("top", p)])
```

```
print("Case Insensitive Match:")  
print([p for p in products if re.search("mouse", p, re.I)])
```

Explanation:

- Uses Regex to perform exact, prefix, suffix, partial, and case-insensitive searches.
- Displays all matching product names.
- Improves product searching in an e-commerce application.

OUTPUT:



```
File Edit Format Shell Debug Options Window Help
Python Shell 3.12.0
File Edit Format Shell Debug Options Window Help
Python Shell 3.12.0

>>> exact_match = [p for p in products if re.fullmatch("Laptop", p)]
>>> prefix_match = [p for p in products if re.match("La", p)]
>>> suffix_match = [p for p in products if re.search("Bag$", p)]
>>> partial_match = [p for p in products if re.search("top", p)]
>>> case_match = [p for p in products if re.search("mouse", p, re.I)]
>>> print("Exact Match:")
>>> print([p for p in exact_match])
>>> print("Prefix Match:")
>>> print([p for p in prefix_match])
>>> print("Suffix Match:")
>>> print([p for p in suffix_match])
>>> print("Partial Match:")
>>> print([p for p in partial_match])
>>> print("Case Insensitive Match:")
>>> print([p for p in case_match])
>>>
```

```
Python Shell 3.12.0
File Edit Format Shell Debug Options Window Help
Python Shell 3.12.0

>>> exact_match = [p for p in products if re.fullmatch("Laptop", p)]
>>> prefix_match = [p for p in products if re.match("La", p)]
>>> suffix_match = [p for p in products if re.search("Bag$", p)]
>>> partial_match = [p for p in products if re.search("top", p)]
>>> case_match = [p for p in products if re.search("mouse", p, re.I)]
>>> print("Exact Match:")
>>> print([p for p in exact_match])
>>> print("Prefix Match:")
>>> print([p for p in prefix_match])
>>> print("Suffix Match:")
>>> print([p for p in suffix_match])
>>> print("Partial Match:")
>>> print([p for p in partial_match])
>>> print("Case Insensitive Match:")
>>> print([p for p in case_match])
>>>
```

Section 3: Regular Expressions – University Registration System

Code:

```
import re
reg = "2024CSE001"
email = "student@simats.edu.in"
course = "DSA03"
semester = "Semester 5"
mobile = "9876543210"
print("Register Number:", "Valid" if re.fullmatch(r"\d{4}[A-Z]{2,3}\d{3}", reg) else
      "Invalid")
print("Email:", "Valid" if re.fullmatch(r"[A-Za-z0-9._%+-]+@simats\.edu\.in", email)
      else "Invalid")
print("Course Code:", "Valid" if re.fullmatch(r"[A-Z]{3}\d{2}", course) else "Invalid")
print("Semester:", "Valid" if re.fullmatch(r"Semester [1-8]", semester) else "Invalid")
print("Mobile:", "Valid" if re.fullmatch(r"[6-9]\d{9}", mobile) else "Invalid")
if (re.fullmatch(r"\d{4}[A-Z]{2,3}\d{3}", reg) and
    re.fullmatch(r"[A-Za-z0-9._%+-]+@simats\.edu\.in", email) and
    re.fullmatch(r"[A-Z]{3}\d{2}", course) and
    re.fullmatch(r"Semester [1-8]", semester) and
    re.fullmatch(r"[6-9]\d{9}", mobile)):
    print("Registration Successful")
else:
    print("Registration Failed")
```

Explanation:

- Validates register number, institutional email, course code, semester, and mobile number using Regular Expressions.
- Displays validation results for each field.
- If all fields are valid, the registration is successful; otherwise, it fails.

OUTPUT:

```
C:\Users\HP\Desktop> python3 tool.py 3305A0D4E8 CUE123  
C:\Users\HP\Desktop> python3 tool.py 3305A0D4E8 CUE123  
Computer pattern = "^[0-9]{10}$"  
Mobile pattern = "[0-9]{10}$"  
  
reg_valid = re.fullmatch(reg_pattern, register_number)  
email_valid = re.fullmatch(email_pattern, email)  
source_code_valid = re.fullmatch(source_code_pattern, source_code)  
password_valid = re.fullmatch(password_pattern, password)  
mobile_valid = re.fullmatch(mobile_pattern, mobile)  
  
print("University Registration Validation")  
print()  
  
if reg_valid:  
    print("Registered Number Valid")  
else:  
    print("Registered Number Invalid")  
  
if email_valid:  
    print("Email Valid")  
else:  
    print("Email Invalid")  
  
if source_code_valid:  
    print("Source Code Valid")  
else:  
    print("Source Code Invalid")  
  
if password_valid:  
    print("Password Valid")  
else:  
    print("Password Invalid")  
  
if mobile_valid:  
    print("Mobile Number Valid")  
else:  
    print("Mobile Number Invalid")  
  
print()  
  
if reg_valid and email_valid and source_code_valid and password_valid and mobile_  
    print("Registration Successful")  
else:  
    print("Registration Unsuccessful")
```